

THE REAL-WORLD STORY

Imagine looking through a moving train window at a scenic mountain landscape. As the train moves forward 1 step, 1 new tree enters the right side of the window, and 1 old tree exits the left side!

THE SLIDING WINDOW AHA MOMENT

Instead of recomputing the sum/characters of an entire subarray from scratch $O(K)$, we update the running window state in $O(1)$ time:
``windowSum += incoming - outgoing`!`

WHY RECOMPUTING SUBARRAYS FAILS

- **Brute Force $O(N \cdot K)$:** For $N = 100,000$ and $K = 50,000$, requires 5,000,000,000 additions -> TLE Error!
- **Sliding Window $O(N)$:** Each element enters the window once and leaves once. Exactly $2N$ operations total!
- **Auxiliary Space $O(1)$ or $O(K)$:** Reuses frequency map or pointers with minimal extra memory.

2 MAIN WINDOW TYPES

1. Fixed-Size Window (Size K):

[left --- right] (Window size fixed at K)
Slide 1 step: add right element, remove left element.

2. Dynamic Shrinking Window:

Expand right pointer to satisfy condition.
Shrink left pointer while condition is violated!

PREREQUISITES & COMPLEXITY REASONING

Window Variation	Time Complexity	Auxiliary Space
Fixed Window Sum / Max	$O(N)$ single pass	$O(1)$ space
Longest Substring Without Repeating	$O(N)$ single pass	$O(\min(N, M))$ hash set
Minimum Window Substring	$O(N)$ single pass	$O(1)$ char freq array
Sliding Window Maximum (Deque)	$O(N)$ single pass	$O(K)$ monotonic deque

CLUES THAT SCREAM SLIDING WINDOW

1. "Find max/min sum of contiguous subarray of fixed size K" -> Fixed-Size Window
2. "Find longest/shortest contiguous subarray satisfying condition C" -> Dynamic Shrinking Window
3. "Substring containing all characters of pattern T" -> Dynamic Window + Frequency HashMap

PATTERN
1

NO-REPEAT SUBSTRING (Dynamic Unique
Character Window)

e.g. Longest Substring Without Repeating
Characters (LC 3)

WHEN TO USE

Find longest substring without any duplicate
characters in $O(N)$ single pass and $O(\min(N, M))$
space.

TEMPLATE — LC 3

```
public int lengthOfLongestSubstring(String s) {  
    int[] lastIdx = new int[128];  
    Arrays.fill(lastIdx, -1);  
    int left = 0, maxLen = 0;  
    for (int right = 0; right < s.length(); right++) {  
        char c = s.charAt(right);  
        if (lastIdx[c] >= left) {  
            left = lastIdx[c] + 1; // Jump left pointer directly past du  
plicate!  
        }  
        lastIdx[c] = right;  
        maxLen = Math.max(maxLen, right - left + 1);  
    }  
    return maxLen;  
}
```

PATTERN 2

K-FLIP ZERO WINDOW (Max Consecutive Ones III) e.g. Max Consecutive Ones III (LC 1004)

WHEN TO USE

Find longest subsegment of 1s allowing at most K zero flips in O(N) time and O(1) space.

TEMPLATE — LC 1004

```
public int longestOnes(int[] nums, int k) {
    int left = 0, zeroCount = 0, maxLen = 0;
    for (int right = 0; right < nums.length; right++) {
        if (nums[right] == 0) zeroCount++;
        while (zeroCount > k) {
            if (nums[left] == 0) zeroCount--;
            left++;
        }
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

PATTERN 4 PERMUTATION IN STRING (Fixed-Size Frequency Matcher) e.g. Permutation in String (LC 567)**MATCHES COUNTER $O(26)$**

Fixed window of size `s1.length()`. Maintain `matches` count out of 26 letters. Update `matches` in $O(1)$ time on each slide!

TEMPLATE — LC 567

```
public boolean checkInclusion(String s1, String s2) {
    if (s1.length() > s2.length()) return false;
    int[] c1 = new int[26], c2 = new int[26];
    for (int i = 0; i < s1.length(); i++) {
        c1[s1.charAt(i) - 'a']++; c2[s2.charAt(i) - 'a']++;
    }
    int matches = 0;
    for (int i = 0; i < 26; i++) if (c1[i] == c2[i]) matches++;
    for (int l = 0, r = s1.length(); r < s2.length(); l++, r++) {
        if (matches == 26) return true;
        int addIdx = s2.charAt(r) - 'a', delIdx = s2.charAt(l) - 'a';
        c2[addIdx]++;
        if (c1[addIdx] == c2[addIdx]) matches++;
        else if (c1[addIdx] + 1 == c2[addIdx]) matches--;
        c2[delIdx]--;
        if (c1[delIdx] == c2[delIdx]) matches++;
        else if (c1[delIdx] - 1 == c2[delIdx]) matches--;
    }
    return matches == 26;
}
```

PATTERN
5

SLIDING WINDOW MAXIMUM (Monotonic Decreasing Deque)

e.g. Sliding Window Maximum (LC 239)

MONOTONIC DEQUE STRATEGY

Deque stores indices in decreasing order of value. Remove smaller elements from tail before adding new index. Front of deque is ALWAYS the max element in current window!

TEMPLATE — LC 239

```
public int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    int[] res = new int[n - k + 1];
    Deque<Integer> deque = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        // 1. Remove indices outside current window boundaries:
        if (!deque.isEmpty() && deque.peekFirst() < i - k + 1) deque.pollFirst();
        // 2. Maintain monotonic decreasing order in deque:
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) deque.pollLast();
        deque.offerLast(i);
        // 3. Store result once first full window of size K is formed:
        if (i >= k - 1) res[i - k + 1] = nums[deque.peekFirst()];
    }
    return res;
}
```

COMPLETE SLIDING WINDOW PROBLEM LADDER SUMMARY					
LC #	Problem	Pattern	Time	Space	Diff
643	Max Average Subarray I	Fixed-Size Window K	O(N)	O(1)	E
3	Longest Substring Without Repeating	Dynamic Shrinking Set Window	O(N)	O(min(N, M))	M
1004	Max Consecutive Ones III	K-Flip Zero Window	O(N)	O(1)	M
567	Permutation in String	Fixed Frequency Matcher	O(N)	O(1)	M
424	Longest Repeating Replacement	Max Frequency Window	O(N)	O(1)	M
76	Minimum Window Substring	2-Map Matcher	O(N)	O(1)	H
239	Sliding Window Maximum	Monotonic Decreasing Deque	O(N)	O(K)	H

LC 643 · Maximum Average Subarray I

EASY

Pattern: Fixed Window K
Key Insight: `sum += in - out`.

```
public double findMaxAverage(int[] nums, int k) {
    int sum = 0; for (int i = 0; i < k; i++) sum += nums[i];
    int maxS = sum;
    for (int r = k; r < nums.length; r++) {
        sum += nums[r] - nums[r - k];
        maxS = Math.max(maxS, sum);
    }
    return (double) maxS / k;
}
```

LC 3 · Longest Substring Without Repeating

MEDIUM

Pattern: Dynamic Set Window
Key Insight: Shrink `left` on duplicate.

```
public int lengthOfLongestSubstring(String s) {
    Set<Character> set = new HashSet<>();
    int l = 0, maxL = 0;
    for (int r = 0; r < s.length(); r++) {
        while (set.contains(s.charAt(r))) { set.remove(s.charAt(l)); l++; }
        set.add(s.charAt(r)); maxL = Math.max(maxL, r - l + 1);
    }
    return maxL;
}
```

LC 1004 · Max Consecutive Ones III

MEDIUM

Pattern: K-Flip Window
Key Insight: Shrink `l` when `zeros > k`.

```
public int longestOnes(int[] nums, int k) {
    int l = 0, zeros = 0, maxL = 0;
    for (int r = 0; r < nums.length; r++) {
        if (nums[r] == 0) zeros++;
        while (zeros > k) { if (nums[l] == 0) zeros--; l++; }
        maxL = Math.max(maxL, r - l + 1);
    }
    return maxL;
}
```


LC 567 · Permutation in String

MEDIUM

Pattern: Fixed Frequency Match
Key Insight: Match `int[26]` frequency arrays.

```
public boolean checkInclusion(String s1, String s2) {
    if (s1.length() > s2.length()) return false;
    int[] c1 = new int[26], c2 = new int[26];
    for (int i = 0; i < s1.length(); i++) {
        c1[s1.charAt(i) - 'a']++; c2[s2.charAt(i) - 'a']++;
    }
    if (Arrays.equals(c1, c2)) return true;
    for (int l = 0, r = s1.length(); r < s2.length(); l++, r++) {
        c2[s2.charAt(r) - 'a']++; c2[s2.charAt(l) - 'a']--;
        if (Arrays.equals(c1, c2)) return true;
    }
    return false;
}
```

LC 424 · Longest Repeating Char Replacement

MEDIUM

Pattern: Max Frequency Window
Key Insight: `(r - l + 1) - maxFreq > k`.

```
public int characterReplacement(String s, int k) {
    int[] count = new int[26];
    int l = 0, maxF = 0, maxL = 0;
    for (int r = 0; r < s.length(); r++) {
        maxF = Math.max(maxF, ++count[s.charAt(r) - 'A']);
        while ((r - l + 1) - maxF > k) {
            count[s.charAt(l) - 'A']--; l++;
        }
        maxL = Math.max(maxL, r - l + 1);
    }
    return maxL;
}
```

LC 76 · Minimum Window Substring

HARD

Pattern: 2-Map Matcher
Key Insight: Shrink `l` while `formed == required`.

```
public String minWindow(String s, String t) {
    int[] map = new int[128];
    for (char c : t.toCharArray()) map[c]++;
    int l = 0, r = 0, count = t.length(), minL = Integer.MAX_VALUE, start = 0;
    while (r < s.length()) {
        if (map[s.charAt(r++)]-- > 0) count--;
        while (count == 0) {
            if (r - l < minL) { minL = r - l; start = l; }
            if (map[s.charAt(l++)]++ == 0) count++;
        }
    }
    return minL == Integer.MAX_VALUE ? "" : s.substring(start, start + minL);
}
```

LC 239 · Sliding Window Maximum

HARD

Pattern: Monotonic Deque
Key Insight: Deque front stores window max index.

```
public int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    int[] res = new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        if (!dq.isEmpty() && dq.peekFirst() < i - k + 1) dq.pollFirst();
        while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) dq.pollLast();
        dq.offerLast(i);
        if (i >= k - 1) res[i - k + 1] = nums[dq.peekFirst()];
    }
    return res;
}
```

🔍 DRY RUN — Longest Substring Without Repeating ("abcabcb")

Step	right (char)	set before	Action	set after	maxLen
1	0 ('a')	{}	Add 'a'	{'a'}	1
2	1 ('b')	{'a'}	Add 'b'	{'a', 'b'}	2
3	2 ('c')	{'a', 'b'}	Add 'c'	{'a', 'b', 'c'}	3
4	3 ('a')	{'a', 'b', 'c'}	Duplicate 'a'! Shrink left past 'a'	{'b', 'c', 'a'}	3 ✓

📐 MATHEMATICAL PROOF — O(N) LINEAR RUNTIME OF DYNAMIC WINDOW

Claim: Dynamic Sliding Window runs in exact O(N) total time despite nested `while` loop.

Proof: The `right` pointer increments N times. The `left` pointer increments at most N times across the entire execution. Since `left` never moves backward, total operations $\leq 2N \rightarrow$ **Exact O(N) Linear Time!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Write Fixed Window K template in 2 minutes
- ☐ Code Longest Substring Without Repeating in O(N)
- ☐ Write Max Consecutive Ones III k-flip template
- ☐ Code Permutation in String with `int[26]` match
- ☐ Code Minimum Window Substring in 10 minutes
- ☐ Write Sliding Window Maximum with Monotonic Deque
- ☐ Prove why total operations $\leq 2N$ for dynamic window
- ☐ Explain why sliding window fails for negative numbers

👛 GOLDEN RULES — NEVER FORGET

- Rule 1 — Expand Right, Shrink Left**

In dynamic window problems, ALWAYS expand `right` pointer first to incorporate new element, then shrink `left` pointer inside a `while` loop until window condition becomes valid again!
- Rule 2 — Use Deque Front for Window Maximum**

In Monotonic Deque, store ARRAY INDICES instead of values! Indices allow checking if front element fell out of window range (`deque.peekFirst() < right - k + 1`).